

## **Architecture Selection: We chose the Microservices Architecture for the Hospital Bed & ICU Allocation Dashboard System.**

**Architectural Choice:** We selected the Microservices Architecture, breaking the system down into independently deployable services (e.g., Triage Manager, Real-Time Sync Engine, Ambulance Orchestrator).

### **Two Reasons:**

1. **Fault Isolation for Critical Care:** If the external inter-facility ambulance transfer workflow experiences downtime or connectivity issues, the internal ER bed allocation and triage scoring modules remain fully operational, ensuring patient care is not interrupted.
2. **Technology Diversity:** The system requires different protocols—standard REST APIs for computing triage scores and WebSockets for real-time dashboard updates. Microservices allow us to build the Real-Time Sync Engine using an event-driven framework (like Node.js), while the core database management can utilize structured relational schemas optimized for ACID transactions.

**Security Advantage:** This architecture provides granular, service-level security. The Real-Time Sync Engine, which broadcasts to all connected ER dashboard clients, can be strictly isolated from the Triage Manager. This ensures that dashboard clients receive only anonymized bed occupancy status, while sensitive patient vitals and triage data remain locked behind the Triage Manager's authenticated API boundaries.

**Performance Benefit:** Independent scaling resolves performance bottlenecks. Because bed status transitions must propagate to all clients in under 1 second, the WebSocket Sync Engine can be scaled horizontally during peak ER hours independently of the Database or Ambulance components, ensuring the target latency is met without over-provisioning the entire system.